UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA
Faculty of Engineering, Built Environment and Information Technology

EDC 310

Digital Communications

Practical 3: Report

Group 8

Name and Surname	Student Number	Signature	% Contribution
A. van Niekerk	19007583		50
A.S. Nel	19014432		50

By signing this assignment we confirm that we have read and are aware of the University of Pretoria's policy on academic dishonesty and plagiarism and we declare that the work submitted in this assignment is our own as delimited by the mentioned policies. We explicitly declare that no parts of this assignment have been copied from current or previous students' work or any other sources (including the internet), whether copyrighted or not. We understand that we will be subjected to disciplinary actions should it be found that the work we submit here does not comply with the said policies.

November 27, 2021

Contents

1 Introduction and Aim . . . 2
2 Linear Block Codes . . . 2
2.1 Theoretical background . . . 2
2.2 Design and implementation . . . 3
3 Convolutional Codes . . . 3
3.1 Theoretical background . . . 3
3.2 Design and implementation . . . 4
4 Simulations and Results . . . 6
4.1 Linear Block Codes . . . 6
4.2 Convolutional Codes . . . 7
5 Results . . . 7
5.1 Linear Block Codes . . . 7
5.2 Convolutional Codes . . . 8
6 Discussion . . . 10
6.1 Linear Block Codes . . . 10
6.2 Convolutional Codes . . . 10
7 Conclusion . . . 11

A Simulation Code **13**

B Linear Block Code **14**

C Convolutional Code **17**

D Question 3 Code **29**

List of Figures

3.2.1 Shift register and state diagram for g = [4,6,5] 5
5.1.1 BER as a function of SNR for BPSK with linear block codes and uncoded detection . . 7
5.1.2 BER as a function of SNR for QPSK with linear block codes and uncoded detection . . 8
5.1.3 BER as a function of SNR for 8PSK with linear block codes and uncoded detection . . . 8
5.2.1 BER as a function of SNR for Convolution using BPSK 9
5.2.2 BER as a function of SNR for Convolution using QPSK 9
5.2.3 BER as a function of SNR for Convolution using 8PSK 10

1 Introduction and Aim

Practical 2 will have students develop two simulation platforms that should encode and decode modulated information through an Additive White Gaussian Noise (AWGN) without multipath. The simulation platform that was developed for practical 1 will be used to generate uncoded data blocks of length $N_u = 100$.

Error Correction Coding (ECC) is a process by which information is encoded before modulation and transmission. This encoded information is then decoded on the receiver's end after it has been demodulated and detected. Any errors in the decoded information that may have been picked up during transmission is corrected. The way in which errors are detected and corrected is through the controlled addition of redundant information at the receiver and then using the knowledge of the redundant information to help correct the errors.

The aim of ECC is to reduce the overall amount of errors in transmitted information without the need to increase transmission power, resulting in a higher Signal-to-Noise Ration (SNR). Two well known ECC methods are Linear Block Codes and Convolutional Codes.

The first simulation platform will make use of a variation of Linear Block Codes, low-density parity check (LDPC) code, and the second will use convolutional codes. Simulations will be run on both platforms, for BPSK, QPSK and 8PSK modulation schemes. At the end of the simulations, the Bit Error Rate (BER) must be plotted against the SNR in order to compare the performance of the two encoding schemes.

2 Linear Block Codes

2.1 Theoretical background

Linear block codes are the most basic error correction codes, but a variation of these codes, the LDPC (Low Density Parity Check) code, are used in all modern wireless communication system protocols. This variation will be one of the subjects for Practical 3.

Before information can be transmitted, it must be encoded in a specific manner. Encoding works by multiplying the source bits with a generator matrix G. For an AWGN channel with no multipath:

$$c = sG + n \tag{1.1}$$

where c is the codeword, s is the source information to be transmitted, G is the generator matrix and n is the noise add by the AWGN channel. Information gets transmitted in coded blocks at a rate of $R_c = k/n$.

The generator matrix is set up in a manner dependant on the transmission rate:

$$G = [I_{kxk} P]$$

where I_{kxk} is a kxk identity matrix and P is the matrix consisting of parity bits.

After transmission and reception, information must be decoded. In order to search for error positions, the syndrome vector z must be generated:

$$z = \tilde{c}H^T \tag{1.2}$$

where $\tilde{c}$ is the estimated codeword (potentially with errors) after detection and symbol to bit mapping and H is the parity check matrix.

if no errors were introduced:

$$z = \tilde{c}H^T = 0$$

and $c = \tilde{c}$. If errors were introduced then:

$$z = \tilde{c}H^T \neq 0$$

and the error must be found in the rows of H^T. If the errors can not be found rows in H^T must be found that will produce z if XOR'ed. The columns that give z when XOR'ed are where the errors are and the bits in $\tilde{c}$ corresponding to these positions must be flipped.

The amount of errors that can be corrected by a given code is determined by:

$$t = \frac{d_{min} - 1}{2}$$

where d_{min} is the minimum distance of the code.

2.2 Design and implementation

The simulation platform of practical 1 will be used as the foundation for practical 3, with specific focus on bit generation, noise addition, bit-to-symbol mapping and bit detection.

For the encoding the main algorithm will first generate a number of uncoded bits that must be transmitted. This group of bits is split into smaller groups which are multiplied by the generator matrix $\mathbf{G}$, resulting in double the amount of coded bits from each smaller. These coded bits are subsequently mapped to symbols and transmitted, resulting in the addition of noise.

Once the symbols are transmitted they must be detected and converted to bits using the platform from practical 1. The detected bits are then decoded according to the process described in the previous section in smaller groups. After decoding and correcting any errors, the first half of the bits from each smaller group is taken as the estimated source bits.

Once the source bits have been estimated, they can be compared to the actual source bits in order to calculate the BER. The BER can then finally be plotted as a function of the SNR.

The code has been implemented in such a way so as to specify a number iterations in order to find the average BER over a specific SNR and thus improve the accuracy of the final graphs.

3 Convolutional Codes

3.1 Theoretical background

Convolutional codes have been around since 1955, It was only in 1967 that the Viterbi MLSE algorithm was purposed by Andrew Viterbi to decode convolution codes.

Convolutional codes are powerful error correction codes able to detect and correct burst errors in wireless communication systems. Encoding works by attractively a block of encoded bits by shifting the bits through a shift register, while the contents of the shift register are multiplied as determined by the encoder connections. Decoding is done by using the MLSE algorithm, this algorithm takes the entire path into consideration and results in a lower error rate.

The convolution encoder is a very powerful algorithm and method of that provides redundancy and enhances the reliability of digital signals transmitted over the channel. The linearity of the convolution encoder makes it very efficient when the source data information sent is significantly increased. However, in general, convolutional coding is used for small streams or blocks of data, and linear block coding is used for large amounts of source data.

In contrast to linear block codes, convolution encoders and transmitters send only parity bits or parity symbols over the channel. The source bit is "hidden" within the parity bit. For this reason, the technique used to decode the information on the receiver side from the encoded convolutional code to the original source information is of paramount importance.

The convolutional encoder is characterized by three main parameters: the code rate ($Rc = \frac{n}{k}$), the generator polynomial, and the constraint length (K).

The cost of each nod in the Vitribi MLSE algorithm is determined by the following formula:

$$\Delta_t = \sum_{n=0}^{K} |\tilde{c}_t - c_t|^2 \tag{1.3}$$

3.2 Design and implementation

This practical required students to implement a simulation platform to generate 100 uniform random bits since the N_c was given as 300 coded bits.

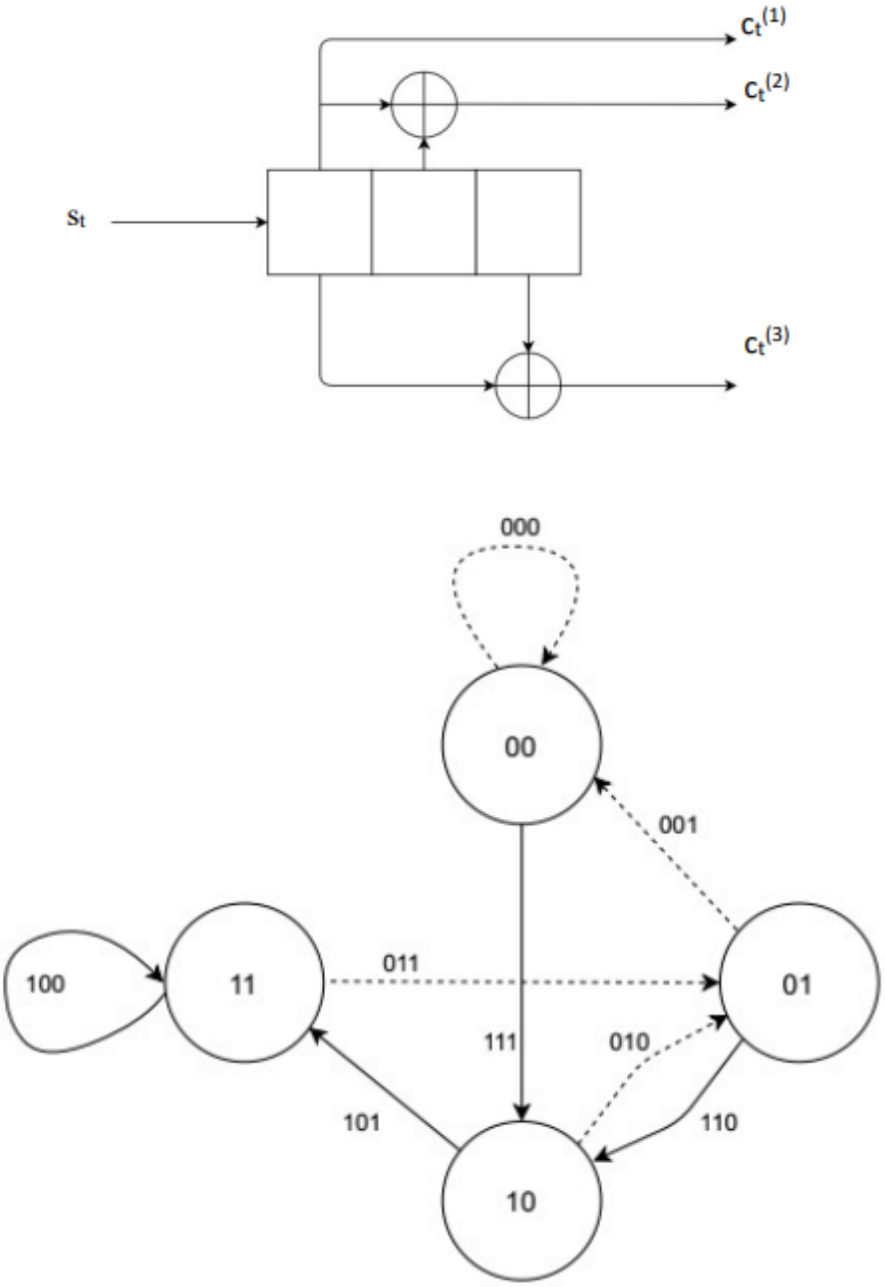

Figure 3.2.1: Shift register and state diagram for g = [4,6,5]

The above shift register in figure 3.2.1 was used since the generator polynomial(g) was given as [4,6,5] and the constraint length(K) as 3.

For this practical a Viterbi Mlse class was implemented that takes in the received bits and g as constructors. A helper class namely MNode was also implemented this class take in the node status as construction parameters, the node status is determined by the shift register.

The Viterbi Mlse class constructs a trellis with the received bits. The constructed node is then stored in a specific place in a 102 by 4 Array that have all been initialized to 9999999999. As the trellis is being built it checks if the value at the specific position, that determined by the Nodes predecessor status and the current nodes status, is equal to 9999999999, if so the created node is set to that position and added to a stack that still needs to be traversed. If the specific position of the Array contains an existing node, the program checks if the cost to get to that node is more than the cost to get to

the current created node. If the costs to get to the node in the array is more, the nodes predecessors appropriate child is set to None and the node is replaced by the newly generated node, The node is also removed from the stack that still needs to be iterated through. The above process will repeat until the Trellis convolutes to the last zero position. This is ensured by appending K-1 0 to the bit array before encoding starts and only shift in zeros at t = length of received - (K-1).

After the trellis is built the node in the farthest right position of the first row will contain the node with the shortest path. This node is then accessed to get the shortest path. The shortest path is obtained by taking taking the first element of the Node's status and appending this to a list, after the bit has been appended the algorithm moves to the current nodes predecessor and repeats itself until the predecessor is equal to None, This means that the first node has been reached. The list containing the first bit of each node in the shortest path is then reversed to obtain the actual path. A new temporary list is created elements from 1 to length of the actual path - 3. This is done to remove the first root Node that starts at 0 and the appended 0. The temp array is then and approximation the actual uncoded data that was initially sent over the communication system.

4 Simulations and Results

To simply the executions of the the simulations a file named Simulation.py was added. This file only contains one function that will run both the Linear Block code and Convolutional code simulations.

4.1 Linear Block Codes

Using practical 1 as the foundation for bit generation, bit-to-symbol mapping and bit detection, the simulation platform for practical 3 could be created. The bit-to-symbol mapping and bit detection algorithms for QPSK was added for the sake of practical 3.

For simulation purposes, 1000 iterations were run for each modulation scheme in order to obtain clearer results with regard to the BER statistics.

For each iteration, 100 bits are generated, which get encoded in smaller groups of 10 by being multiplied with the generator matrix. This results in 20 encoded bits for each group of 10, meaning 200 bits in total will be transmitted. The bits in each group of 20 are mapped to symbols before they are transmitted with the addition of noise.

Once symbols have been received, they are detected and converted back to bits. After decoding and error-correction within the groups of 20, the first 10 bits are used as the estimated bits. These groups of 10 bits are finally joined to a final sequence of 100 total bits, which represents the final estimated sequence.

Now that the bits have been transmitted and received, the BER can be calculated for each SNR and compared to the performance of a regular uncoded optimal detection algorithm and be plotted for visual representation.

It must be noted that for 8PSK to to be mapped to symbols, the number of bits that are transmitted must be a multiple of 3. Also, in order for matrices to be reshaped for the purpose of the linear block codes, another specific amount of bits are required. Therefore a total of 300 bits were generated only for 8PSK, resulting in groups of 30 bits being encoded and transmitted.

4.2 Convolutional Codes

The simulation platform developed in practical one was used, the QPSK modulation type was just added and the noise generation function was modified to match practical 3's requirements. Practical 1's bit_detection() algorithm was modified to return the normal bits as well as the bits in bipolar form.

The BER values obtained for each SNR is averaged using 100 iterations. For each SNR in the range -8 to 6 a 100 bits is generated using practical 1's random Gaussian bit generator. K-1 bits is appended to ensure that the trellis convolutes to state 0. After the bits have been generated and appended the bits are encoded using the register seen in figure 3.2.1 on page 5.

The encoded bits is then converted to symbols using practical 1's bits_to_symbols() function. Noise is then added using the modified add_noise() function from practical 1. After the Noise has been added the symbols are converted to bits using practical 1's bit_detection() function. The bipolar bits is then used to create an instance of the Viterbi_MLSE class. The MLSE_decode() function from the class is called to get the approximated received bits. The approximated recieved bits is then compared to the generated bits by calculating the BER values using practical 1's calculate_BER() function.

To allow comparison of the convolutional code's performance, BPSK, QPSK and 8PSK is used. Each modulation's uncoded bits and encoded bit using convolutional code's BER to SNR is plotted on the same plane.

5 Results

5.1 Linear Block Codes

Below are the results of the simulations for the Linear Block Codes:

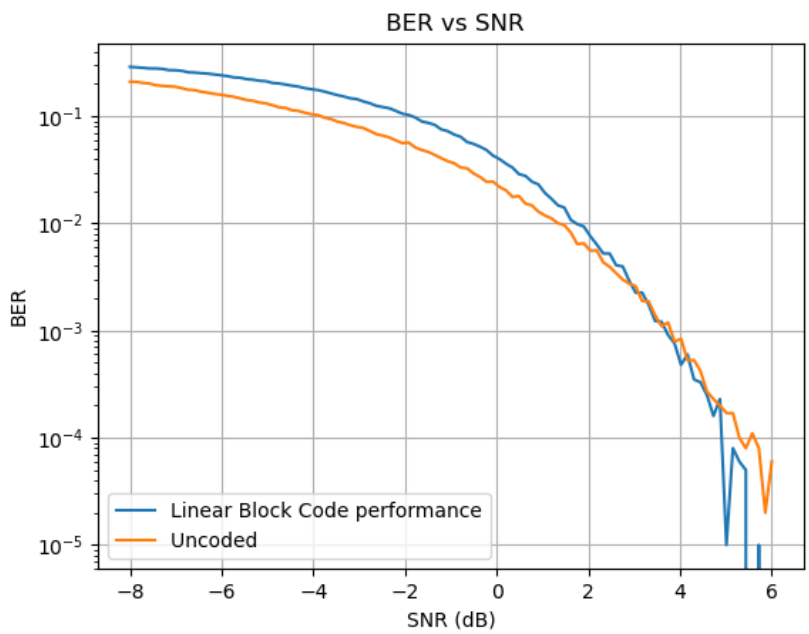

Figure 5.1.1: BER as a function of SNR for BPSK with linear block codes and uncoded detection

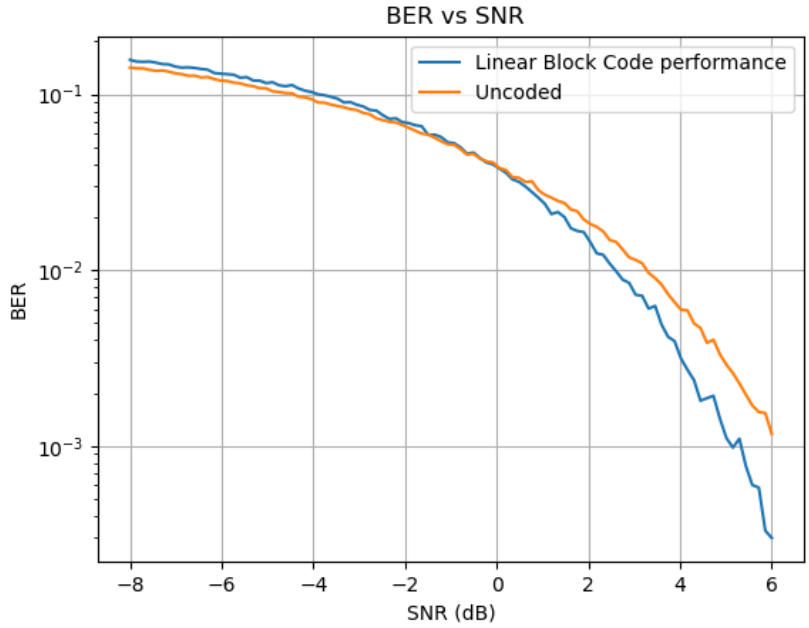

Figure 5.1.2: BER as a function of SNR for QPSK with linear block codes and uncoded detection

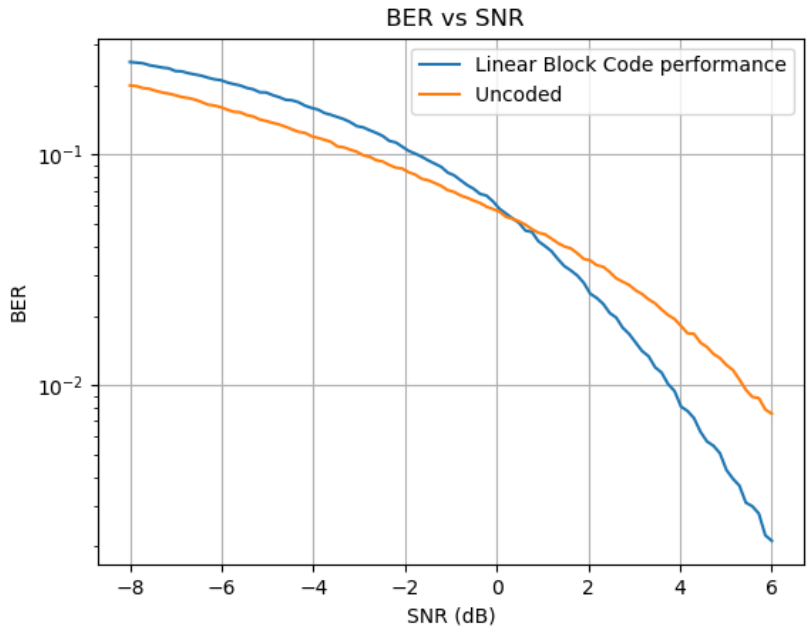

Figure 5.1.3: BER as a function of SNR for 8PSK with linear block codes and uncoded detection

5.2 Convolutional Codes

Below are the results of the simulations for the Convolutional Codes:

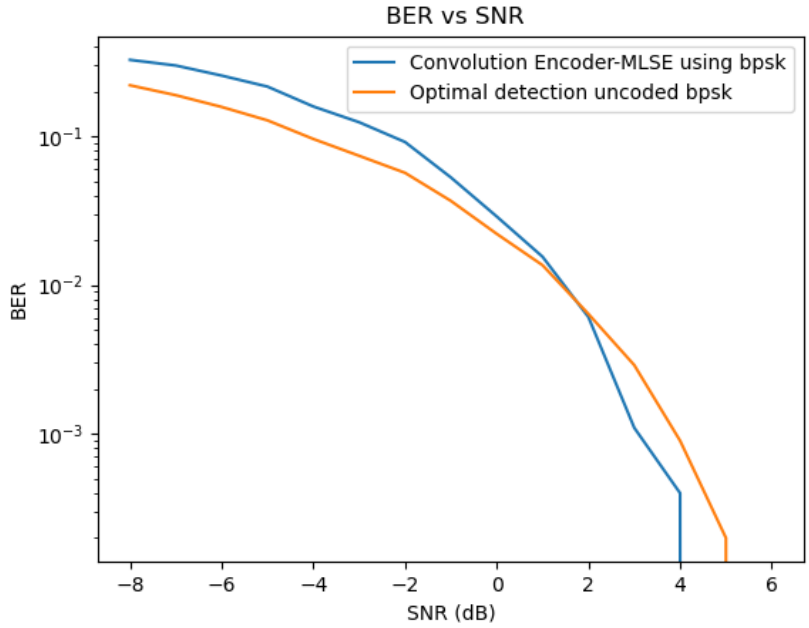

Figure 5.2.1: BER as a function of SNR for Convolution using BPSK

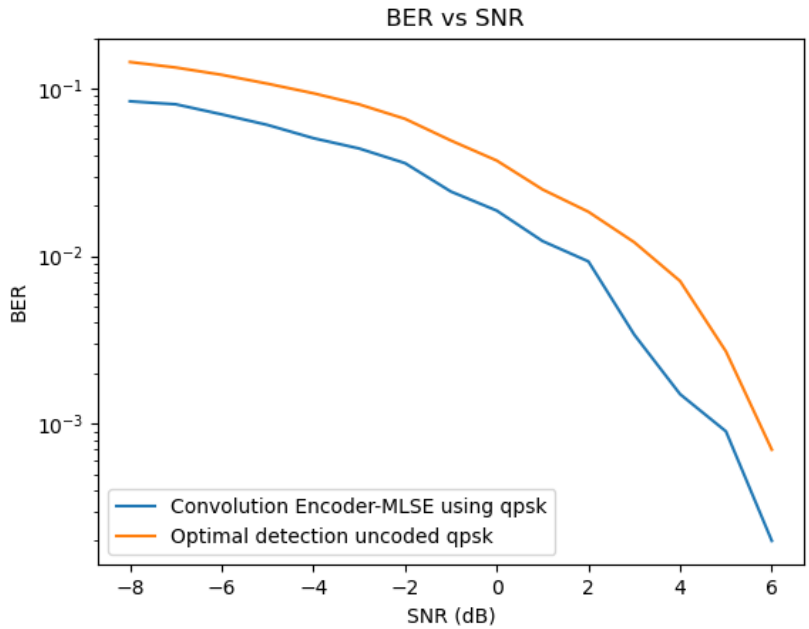

Figure 5.2.2: BER as a function of SNR for Convolution using QPSK

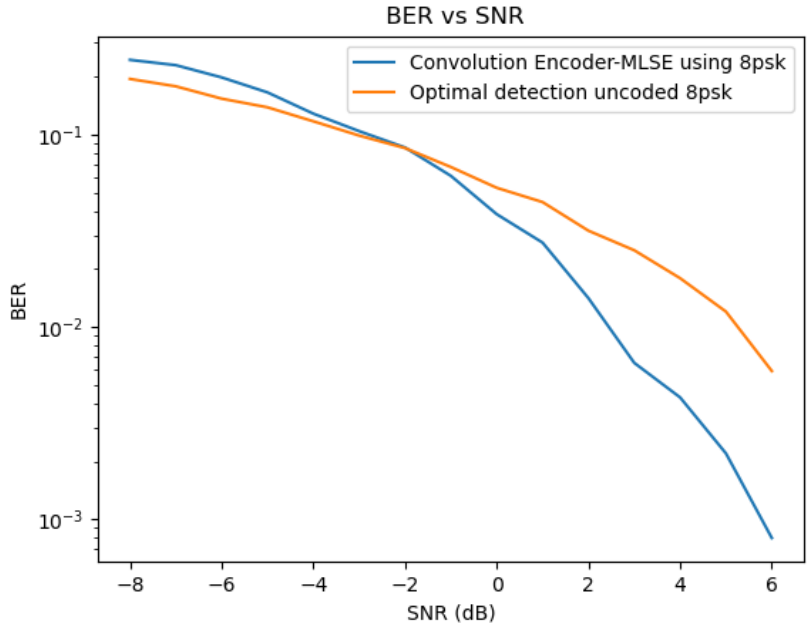

Figure 5.2.3: BER as a function of SNR for Convolution using 8PSK

6 Discussion

6.1 Linear Block Codes

The results will be discussed with reference to figures 5.1.1, 5.1.2 and 5.1.3

When considering the meaning of SNR, a greater SNR results in a more accurate transmission of information. By looking at the development of the graphs for the the linear block codes and the uncoded detection algorithms, a greater SNR results in a lower number of errors.

At lower SNRs, the performance of the uncoded detection algorithm is slightly better than that of the linear block codes. As the SNR increases, the performances of the two methods start to converge. The performance of the linear block codes matches the performance of the uncoded detection algorithm around an SNR of 3 for BPSK, -0.5 for QPSk and 0 for 8PSK.

It can also be seen from the simulations that the BER comes very close to zero around an SNR of 6 while the uncoded detection algorithm lacks in performance slightly. When considering these results alone, it can be said that at low SNR values, uncoded detection will perform slightly better, but eventually the linear block code performance should overtake regular detection as the SNR increases.

6.2 Convolutional Codes

From the results from figures 5.2.1 to 5.2.2 allow us to visualise and compare the performance of Convolutional codes to standard uncoded bits sent across a AWGN channel.

From the results it can be seen that the more iterations ran the smoother the graphs are. The smoothness of the graph increases the reliability of the results.

It can be seen that the Convolutional code greatly reduces the SNR needed to minimize the BER.

This can be seen in figures 5.2.1 to 5.2.3 by comparing the cutoff points on the x-axis of each graph. Although in figure 5.2.1 and figure 5.2.3 it can be seen that the convolutional encoder initially start at a higher BER the rate of which the BER decreases to the increase of SNR is much steeper compared to the uncoded bits.

The steep decrease in BER is due to the fact the Viterbi MLSE algorith is used to detect the encoded bits, this algorithm takes the entire plane of bits into consideration when determining the best path, where as the normal uncoded bit detection only takes a single bit in the plane into consideration.

7 Conclusion

The simulation platform was successfully developed and the performances of ECC methods linear block coding and convolutional coding were investigated.

From the results discussed earlier, it can be seen that ECC do not provide much of an advantage over uncoded detection processes at very low SNRs, yet do outperform the uncoded detection processes at higher SNRs. Since a higher SNR translates to a higher amount of power usage, this would mean that ECC is more efficient at bit detection for higher power outputs than uncoded detection would be.

When it comes to choosing between the two ECC methods, one must consider their overall performances and their complexity. Both linear block codes and convolutional codes perform better than uncoded detection at higher SNRs, but worse at lower SNRs. Thus, if the SNR is high, encoding would provide an advantage. Their performance overall when compare to each other are very similar with no specific method overtaking the other in any considerable amount. The computational complexity of convolutional codes is however considerebly higher than linear block codes.

In conclusion, if a single method had to be used, it would depend on resources relating to power for the SNR and computational resources.

Bibliography

[1] M. S. John Proakis, *Digital Communications, 5th Edition*, 5th ed. McGraw-Hill Science Engineering Math, 2007. [Online]. Available: http://gen.lib.rus.ec/book/index.php?md5=0d60b29a68495e82cdafb3dc7708ff0a

Appendix A

Simulation Code

```
import numpy as np
from CONV import tester
from LBC import run

#Function to run all simulations
def simulate():
    #Linear Block code simulations
    run('bpsk',1000)
    run('qpsk', 1000)
    run('8psk', 1000)
    #Convolutional code simulations
    tester('bpsk')
    tester('qpsk')
    tester('8psk')

simulate()
```

Appendix B

Linear Block Code

```python
import numpy as np
import matplotlib.pyplot as plt
from question_3 import q3
from itertools import combinations

# number of uncoded bits to generate
Nu = 100

# Generator matrix
Gen_mat = [[1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0],
        [0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1],
        [0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0],
        [0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 1],
        [0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 1, 1, 0],
        [0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1],
        [0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1],
        [0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 1]]

# Function to generate parity check matrix
def parity(G):
    GM = np.array(G)
    parity = GM[0:GM.shape[0], GM.shape[0]:GM.shape[1]]

    HM = np.concatenate((parity.transpose(), np.identity(GM.shape[1] -
                GM.shape[0])), 1)

    return HM


# Function to encode sequences of bits
def CW(transmitted, G):  # sent is a list, G is a list, n is noise
    GM = np.array(G)
    s = np.array(transmitted).reshape((-1, GM.shape[0]))
    CWs = s.dot(GM) % 2
    return CWs.flatten()


# Function to decode sequences of bits
def decode(RB, G):  # G is a list and Rec is a list too
    G = np.array(G)
    cest = np.array(RB).reshape((-1, G.shape[1]))
    H = parity(G)
    syndrome = (cest.dot(H.T)) % 2

    for p in range(0, len(cest)):
        if (sum(syndrome[p] >= 1)):
            index = []
            for i in H.T:
                if ((i == syndrome[p]).all()):
                    index.append(1)
                else:
                    index.append(0)
            if (sum(index) > 0):
                cest[p] ^= np.array(index)
            else:
                for c in combinations(range(len(H.T)), 2):
                    TO_XOR = H.T[c[0]].astype(int) ^ H.T[c[1]].astype(int)
                    if (TO_XOR == syndrome[p]).all():
                        cest[p][c[0]] ^= 1
                        cest[p][c[1]] ^= 1
                        break
    return cest[:,:G.shape[0]].flatten()


# Main function that executes whole algorithm and plots the results
# Takes the modulation scheme and number of iterations as input variables
def run(modulation,iterations):
    xs = np.linspace(-8, 6, 100)
    yy = []
    yy1 = []

```

```python
    for p in range(iterations):
        yvalues = []
        yvalues1 = []
        # BPSK
        if modulation == 'bpsk':
            q = q3(Nu)
            bits = q.generate_N_bits()
            for x in xs:
                BER_coded = 0
                BER_uncoded = 0
                main = []
                decoded = []
                for i in range(10):
                    temp = []
                    for j in range(10):
                        temp.append(bits[i*10 +j])
                    main = [*main,*temp]

                coded_transmitted,symbol = q.bit_detection(
                    q.add_noise(q.bits_to_symbols(list(CW(main,Gen_mat)),
                                                'bpsk'),x,q.get_fbits('bpsk'),
                                                0.5),'bpsk')
                uncoded_transmitted, symbol1 = q.bit_detection((
                    q.add_noise(q.bits_to_symbols(bits,'bpsk'),x,
                                                q.get_fbits('bpsk'), 1)),'bpsk')
                for i in range(10):
                    temp = []
                    for j in range(20):
                        temp.append(coded_transmitted[i*20 +j])
                    tmp = list(decode(temp, Gen_mat))
                    c = []
                    for k in range(10):
                        c.append(tmp[k])
                    decoded = [*decoded,*c]
                BER_coded += q.calculate_BER(bits, decoded)
                BER_uncoded += q.calculate_BER(bits, uncoded_transmitted)
                np.array(yvalues.append(BER_coded))
                np.array(yvalues1.append(BER_uncoded))
            yy.append(yvalues)
            yy1.append(yvalues1)

        # QPSK
        if modulation == 'qpsk':
            q = q3(Nu)
            bits = q.generate_N_bits()
            for x in xs:
                BER_coded = 0
                BER_uncoded = 0
                main = []
                decoded = []
                for i in range(10):
                    temp = []
                    for j in range(10):
                        temp.append(bits[i * 10 + j])
                    main = [*main, *temp]
                coded_transmitted, symbol = q.bit_detection(q.add_noise
                                (q.bits_to_symbols(list(CW(main,
                                Gen_mat)), 'qpsk'), x,
                                 q.get_fbits('qpsk'), 0.5), 'qpsk')
                uncoded_transmitted, symbol1 = q.bit_detection((
                    q.add_noise(q.bits_to_symbols(bits,'qpsk'), x,
                                q.get_fbits('qpsk'), 1)), 'qpsk')
                for i in range(10):
                    temp = []
                    for j in range(20):
                        temp.append(coded_transmitted[i * 20 + j])
                    tmp = list(decode(temp, Gen_mat))
                    c = []
                    for k in range(10):
                        c.append(tmp[k])
                    decoded = [*decoded, *c]
                BER_coded += q.calculate_BER(bits, decoded)
                BER_uncoded += q.calculate_BER(bits, uncoded_transmitted)
                np.array(yvalues.append(BER_coded))
                np.array(yvalues1.append(BER_uncoded))
            yy.append(yvalues)
            yy1.append(yvalues1)
        # 8PSK
        if modulation == '8psk':
            q = q3(300)
            bits = q.generate_N_bits()
            for x in xs:
                BER_coded = 0
                BER_uncoded = 0
                main = []
                decoded = []
                for i in range(10):
                    temp = []
                    for j in range(30):
                        temp.append(bits[i * 30 + j])
                    main = [*main, *temp]
                coded_transmitted, symbol = q.bit_detection(q.add_noise
                                (q.bits_to_symbols(list(CW(main, Gen_mat))
                                ,'8psk'), x, q.get_fbits('8psk'), 0.5),
```

```
                    '8psk')
                uncoded_transmitted, symbol1 = q.bit_detection(
                    (q.add_noise(q.bits_to_symbols(bits,'8psk'),
                                x, q.get_fbits('8psk'), 1)), '8psk')
                for i in range(10):
                    temp = []
                    for j in range(60):
                        temp.append(coded_transmitted[i * 60 + j])
                    tmp = list(decode(temp, Gen_mat))
                    c = []
                    for k in range(30):
                        c.append(tmp[k])
                    decoded = [*decoded, *c]

                BER_coded += q.calculate_BER(bits, decoded)
                BER_uncoded += q.calculate_BER(bits, uncoded_transmitted)
                np.array(yvalues.append(BER_coded))
                np.array(yvalues1.append(BER_uncoded))
            yy.append(yvalues)
            yy1.append(yvalues1)

    plt.semilogy(xs, sum(np.array(yy))/iterations,
                 label="Linear Block Code performance")
    plt.ylabel('BER')
    plt.xlabel('SNR (dB)')
    plt.semilogy(xs, sum(np.array(yy1))/iterations,
                 label="Uncoded ")
    plt.grid()
    plt.title(" BER vs SNR")
    plt.legend()
    plt.show()
```

Appendix C

Convolutional Code

```python
import numpy as np
from copy import deepcopy
from collections import deque
import matplotlib.pyplot as plt
from numpy.core.fromnumeric import transpose
from numpy.core.records import array
from numpy.lib.function_base import append, delete
from question_3 import q3

K=3

class MNode:
    def __init__(self, status):
        self.status = status
        self.heuristic = None
        self.Predecessor = None
        self.child = [None, None]
        self.time = -1


    #Function to return the appropriate c values to computate the cost
    def getCval(self, state, currstate):
        temp = []
        if(state == [0,0] and currstate == [0,0]):
            temp.append(-1)
            temp.append(-1)
            temp.append(-1)
            return temp
        elif(state == [0,0] and currstate == [1,0]):
            temp.append(1)
            temp.append(1)
            temp.append(1)
            return temp
        elif(state == [1,0] and currstate == [0,1]):
            temp.append(-1)
            temp.append(1)
            temp.append(-1)
            return temp
        elif(state == [1,0] and currstate == [1,1]):
            temp.append(1)
            temp.append(-1)
            temp.append(1)
            return temp
        elif(state == [1,1] and currstate == [1,1]):
            temp.append(1)
            temp.append(-1)
            temp.append(-1)
            return temp
        elif(state == [1,1] and currstate == [0,1]):
            temp.append(-1)
            temp.append(1)
            temp.append(1)
            return temp
        elif(state == [0,1] and currstate == [1,0]):
            temp.append(1)
            temp.append(1)
            temp.append(-1)
            return temp
        elif(state == [0,1] and currstate == [0,0]):
            temp.append(-1)
            temp.append(-1)
            temp.append(1)
            return temp



class Viterbi_MLSE:
    def __init__(self, received, G):
        self.K = len(G)
        self.received = np.array(received).reshape((-1, self.K))
        self.BlockN = len(self.received)
        self.To_traverse = []
        self.Root = None
        self.path = []
        self.cost = 999999999999
        rows, cols = (4, len(received))
        self.array = [[999999999999 for i in range(cols)] for j in range(rows)]


    #Function that shift the new bits in
    def ShiftRegister(self, x_state, m):
        helper = []
        helper.append(m)
```

```
        helper.append(x_state[0])
        return helper


    #Function to get shortest path
    def getPath(self):
        reversed_bits = []
        node = self.array[0][len(self.received)-1]
        self.genPath(node, reversed_bits)
        unreversed_bits = reversed_bits[::-1]
        return unreversed_bits

    def genPath(self,Node, reversed_bits):
        if Node != None:
            reversed_bits.append(Node.status[0])
            self.genPath(Node.Predecessor, reversed_bits)


    #Function to Build the trellis to determine shotest path
    def buildTrellis(self, NodeTotraverse):
        Temp_node_0 = MNode(self.ShiftRegister(NodeTotraverse.status, 0))
        Temp_node_0.time = NodeTotraverse.time + 1
        Temp_node_1 = MNode(self.ShiftRegister(NodeTotraverse.status, 1))
        Temp_node_1.time = NodeTotraverse.time + 1

        #Shifts in both 0's and 1's into the register
        if(Temp_node_0.time < len(self.received)-2):
            heuristic = NodeTotraverse.heuristic + self.calculate_cost(self.received[Temp_node_0.time],Temp_node_0.getCval(NodeTotraverse.status, Temp_node_0.status))
            Temp_node_0.Predecessor = NodeTotraverse
            Temp_node_0.heuristic = heuristic
            heuristic1 = NodeTotraverse.heuristic + self.calculate_cost(self.received[Temp_node_1.time],Temp_node_1.getCval(NodeTotraverse.status, Temp_node_1.status))
            Temp_node_1.Predecessor = NodeTotraverse
            Temp_node_1.heuristic = heuristic1
            Canpass_1 = False
            Canpass_0 = False
            if(Temp_node_0.status == [0,0]):
                if(self.array[0][Temp_node_0.time] == 999999999999):
                    self.array[0][Temp_node_0.time] = Temp_node_0
                    Canpass_0 = True

                elif(heuristic < self.array[0][Temp_node_0.time].heuristic):
                    node_to_change = self.array[0][Temp_node_0.time]
                    to_change_predasessor = node_to_change.Predecessor

                    if(to_change_predasessor.status == [0,0] and node_to_change.status == [0,0]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [0,0] and node_to_change.status == [1,0]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [1,0] and node_to_change.status == [0,1]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [1,0] and node_to_change.status == [1,1]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [1,1] and node_to_change.status == [0,1]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [1,1] and node_to_change.status == [1,1]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [0,1] and node_to_change.status == [0,0]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [0,1] and node_to_change.status == [1,0]):
                        to_change_predasessor.child[1] = None

                    self.array[0][Temp_node_0.time] = Temp_node_0

                    for i in range(len(self.To_traverse)):
                        if(self.To_traverse[i] == node_to_change):
                            self.To_traverse.pop(i)
                            break
                    Canpass_0 = True

            elif(Temp_node_0.status == [0,1]):
```

```
                if(self.array[1][Temp_node_0.time] == 999999999999):
                    self.array[1][Temp_node_0.time] = Temp_node_0
                    Canpass_0 = True
                elif(heuristic < self.array[1][Temp_node_0.time].heuristic):
                    node_to_change = self.array[1][Temp_node_0.time]
                    to_change_predasessor = node_to_change.Predecessor

                    if(to_change_predasessor.status == [0,0] and node_to_change.status == [0,0]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [0,0] and node_to_change.status == [1,0]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [1,0] and node_to_change.status == [0,1]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [1,0] and node_to_change.status == [1,1]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [1,1] and node_to_change.status == [0,1]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [1,1] and node_to_change.status == [1,1]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [0,1] and node_to_change.status == [0,0]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [0,1] and node_to_change.status == [1,0]):
                        to_change_predasessor.child[1] = None

                    self.array[1][Temp_node_0.time] = Temp_node_0
                    for i in range(len(self.To_traverse)):
                        if(self.To_traverse[i] == node_to_change):
                            self.To_traverse.pop(i)
                            break
                    Canpass_0 = True

            elif(Temp_node_0.status == [1,0]):
```

```
                if(self.array[2][Temp_node_0.time] == 999999999999):
                    self.array[2][Temp_node_0.time] = Temp_node_0
                    Canpass_0 = True
                elif(heuristic < self.array[2][Temp_node_0.time].heuristic):
                    node_to_change = self.array[2][Temp_node_0.time]
                    to_change_predasessor = node_to_change.Predecessor

                    if(to_change_predasessor.status == [0,0] and node_to_change.status == [0,0]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [0,0] and node_to_change.status == [1,0]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [1,0] and node_to_change.status == [0,1]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [1,0] and node_to_change.status == [1,1]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [1,1] and node_to_change.status == [0,1]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [1,1] and node_to_change.status == [1,1]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [0,1] and node_to_change.status == [0,0]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [0,1] and node_to_change.status == [1,0]):
                        to_change_predasessor.child[1] = None

                    self.array[2][Temp_node_0.time] = Temp_node_0
                    for i in range(len(self.To_traverse)):
                        if(self.To_traverse[i] == node_to_change):
                            self.To_traverse.pop(i)
                            break
                    Canpass_0 = True

            elif(Temp_node_0.status == [1,1]):
```

```
                if(self.array[3][Temp_node_0.time] == 999999999999):
                    self.array[3][Temp_node_0.time] = Temp_node_0
                    Canpass_0 = True
                elif(heuristic < self.array[3][Temp_node_0.time].heuristic):
                    node_to_change = self.array[3][Temp_node_0.time]
                    to_change_predasessor = node_to_change.Predecessor

                    if(to_change_predasessor.status == [0,0] and node_to_change.status == [0,0]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [0,0] and node_to_change.status == [1,0]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [1,0] and node_to_change.status == [0,1]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [1,0] and node_to_change.status == [1,1]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [1,1] and node_to_change.status == [0,1]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [1,1] and node_to_change.status == [1,1]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [0,1] and node_to_change.status == [0,0]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [0,1] and node_to_change.status == [1,0]):
                        to_change_predasessor.child[1] = None

                    self.array[3][Temp_node_0.time] = Temp_node_0
                    for i in range(len(self.To_traverse)):
                        if(self.To_traverse[i] == node_to_change):
                            self.To_traverse.pop(i)
                            break
                    Canpass_0 = True

            if(Temp_node_1.status == [0,0]):
                if(self.array[0][Temp_node_1.time] == 999999999999):
                    self.array[0][Temp_node_1.time] = Temp_node_1
                    Canpass_1 = True
                elif(heuristic1 < self.array[0][Temp_node_1.time].heuristic):
                    node_to_change = self.array[0][Temp_node_1.time]
                    to_change_predasessor = node_to_change.Predecessor

                    if(to_change_predasessor.status == [0,0] and node_to_change.status == [0,0]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [0,0] and node_to_change.status == [1,0]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [1,0] and node_to_change.status == [0,1]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [1,0] and node_to_change.status == [1,1]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [1,1] and node_to_change.status == [0,1]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [1,1] and node_to_change.status == [1,1]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [0,1] and node_to_change.status == [0,0]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [0,1] and node_to_change.status == [1,0]):
                        to_change_predasessor.child[1] = None

                    self.array[0][Temp_node_1.time] = Temp_node_1
                    for i in range(len(self.To_traverse)):
                        if(self.To_traverse[i] == node_to_change):
                            self.To_traverse.pop(i)
                            break
                    Canpass_1 = True

            elif(Temp_node_1.status == [0,1]):
```

```
                if(self.array[1][Temp_node_1.time] == 999999999999):
                    self.array[1][Temp_node_1.time] = Temp_node_1
                    Canpass_1 = True
                elif(heuristic1 < self.array[1][Temp_node_1.time].heuristic):
                    node_to_change = self.array[1][Temp_node_1.time]
                    to_change_predasessor = node_to_change.Predecessor

                    if(to_change_predasessor.status == [0,0] and node_to_change.status == [0,0]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [0,0] and node_to_change.status == [1,0]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [1,0] and node_to_change.status == [0,1]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [1,0] and node_to_change.status == [1,1]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [1,1] and node_to_change.status == [0,1]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [1,1] and node_to_change.status == [1,1]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [0,1] and node_to_change.status == [0,0]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [0,1] and node_to_change.status == [1,0]):
                        to_change_predasessor.child[1] = None

                    self.array[1][Temp_node_1.time] = Temp_node_1
                    for i in range(len(self.To_traverse)):
                        if(self.To_traverse[i] == node_to_change):
                            self.To_traverse.pop(i)
                            break
                    Canpass_1 = True

            elif(Temp_node_1.status == [1,0]):
                if(self.array[2][Temp_node_1.time] == 999999999999):
                    self.array[2][Temp_node_1.time] = Temp_node_1
                    Canpass_1 = True

                elif(heuristic1 < self.array[2][Temp_node_1.time].heuristic):
                    node_to_change = self.array[2][Temp_node_1.time]
                    to_change_predasessor = node_to_change.Predecessor

                    if(to_change_predasessor.status == [0,0] and node_to_change.status == [0,0]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [0,0] and node_to_change.status == [1,0]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [1,0] and node_to_change.status == [0,1]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [1,0] and node_to_change.status == [1,1]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [1,1] and node_to_change.status == [0,1]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [1,1] and node_to_change.status == [1,1]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [0,1] and node_to_change.status == [0,0]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [0,1] and node_to_change.status == [1,0]):
                        to_change_predasessor.child[1] = None

                    self.array[2][Temp_node_1.time] = Temp_node_1
                    for i in range(len(self.To_traverse)):
                        if(self.To_traverse[i] == node_to_change):
                            self.To_traverse.pop(i)
                            break
                    Canpass_1 = True

            elif(Temp_node_1.status ==
```

```
[1,1]):
                if(self.array[3][Temp_node_1.time] == 999999999999):
                    self.array[3][Temp_node_1.time] = Temp_node_1
                    Canpass_1 = True
                elif(heuristic1 < self.array[3][Temp_node_1.time].heuristic):
                    node_to_change = self.array[3][Temp_node_1.time]
                    to_change_predasessor = node_to_change.Predecessor

                    if(to_change_predasessor.status == [0,0] and node_to_change.status == [0,0]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [0,0] and node_to_change.status == [1,0]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [1,0] and node_to_change.status == [0,1]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [1,0] and node_to_change.status == [1,1]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [1,1] and node_to_change.status == [0,1]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [1,1] and node_to_change.status == [1,1]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [0,1] and node_to_change.status == [0,0]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [0,1] and node_to_change.status == [1,0]):
                        to_change_predasessor.child[1] = None

                    self.array[3][Temp_node_1.time] = Temp_node_1
                    for i in range(len(self.To_traverse)):
                        if(self.To_traverse[i] == node_to_change):
                            self.To_traverse.pop(i)
                            break
                    Canpass_1 = True

            if(Canpass_0 == True):
                self.To_traverse.append(Temp_node_0)
                NodeTotraverse.child[0] = Temp_node_0

            if(Canpass_1 == True):
                self.To_traverse.append(Temp_node_1)
                NodeTotraverse.child[1] = Temp_node_1

        #Start to shift in only 0's into the register
        elif(Temp_node_0.time >= len(self.received)-2 and Temp_node_0.time < len(self.received)-1):
            heuristic = NodeTotraverse.heuristic + self.calculate_cost(self.received[Temp_node_0.time],Temp_node_0.getCval(NodeTotraverse.status, Temp_node_0.status))
            Temp_node_0.Predecessor = NodeTotraverse
            Temp_node_0.heuristic = heuristic

            Canpass_0 = False
            if(Temp_node_0.status == [0,0]):
                if(self.array[0][Temp_node_0.time] == 999999999999):
                    self.array[0][Temp_node_0.time] = Temp_node_0
                    Canpass_0 = True
                elif(heuristic < self.array[0][Temp_node_0.time].heuristic):
                    node_to_change = self.array[0][Temp_node_0.time]
                    to_change_predasessor = node_to_change.Predecessor

                    if(to_change_predasessor.status == [0,0] and node_to_change.status == [0,0]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [0,0] and node_to_change.status == [1,0]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [1,0] and node_to_change.status == [0,1]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [1,0] and node_to_change.status == [1,1]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [1,1] and node_to_change.status == [0,1]):
                        to_change_predasessor.child[0] = None
```

```
                    elif(to_change_predasessor.status == [1,1] and node_to_change.status == [1,1]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [0,1] and node_to_change.status == [0,0]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [0,1] and node_to_change.status == [1,0]):
                        to_change_predasessor.child[1] = None

                    self.array[0][Temp_node_0.time] = Temp_node_0
                    for i in range(len(self.To_traverse)):
                        if(self.To_traverse[i] == node_to_change):
                            self.To_traverse.pop(i)
                            break
                    Canpass_0 = True
            elif(Temp_node_0.status == [0,1]):
                if(self.array[1][Temp_node_0.time] == 999999999999):
                    self.array[1][Temp_node_0.time] = Temp_node_0
                    Canpass_0 = True
                elif(heuristic < self.array[1][Temp_node_0.time].heuristic):
                    node_to_change = self.array[1][Temp_node_0.time]
                    to_change_predasessor = node_to_change.Predecessor

                    if(to_change_predasessor.status == [0,0] and node_to_change.status == [0,0]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [0,0] and node_to_change.status == [1,0]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [1,0] and node_to_change.status == [0,1]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [1,0] and node_to_change.status == [1,1]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [1,1] and node_to_change.status == [0,1]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [1,1] and node_to_change.status == [1,1]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [0,1] and node_to_change.status == [0,0]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [0,1] and node_to_change.status == [1,0]):
                        to_change_predasessor.child[1] = None

                    self.array[1][Temp_node_0.time] = Temp_node_0
                    for i in range(len(self.To_traverse)):
                        if(self.To_traverse[i] == node_to_change):
                            self.To_traverse.pop(i)
                            break
                    Canpass_0 = True

            elif(Temp_node_0.status == [1,0]):
                if(self.array[2][Temp_node_0.time] == 999999999999):
                    self.array[2][Temp_node_0.time] = Temp_node_0
                    Canpass_0 = True
                elif(heuristic < self.array[2][Temp_node_0.time].heuristic):
                    node_to_change = self.array[2][Temp_node_0.time]
                    to_change_predasessor = node_to_change.Predecessor

                    if(to_change_predasessor.status == [0,0] and node_to_change.status == [0,0]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [0,0] and node_to_change.status == [1,0]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [1,0] and node_to_change.status == [0,1]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [1,0] and node_to_change.status == [1,1]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [1,1] and node_to_change.status == [0,1]):
                        to_change_predasessor.child[0] = None
                    elif(
```

```
to_change_predasessor.status == [1,1] and node_to_change.status == [1,1]):
                    to_change_predasessor.child[1] = None
                elif(to_change_predasessor.status == [0,1] and node_to_change.status == [0,0]):
                    to_change_predasessor.child[0] = None
                elif(to_change_predasessor.status == [0,1] and node_to_change.status == [1,0]):
                    to_change_predasessor.child[1] = None

                self.array[2][Temp_node_0.time] = Temp_node_0
                for i in range(len(self.To_traverse)):
                    if(self.To_traverse[i] == node_to_change):
                        self.To_traverse.pop(i)
                        break
                Canpass_0 = True

        elif(Temp_node_0.status == [1,1]):
            if(self.array[3][Temp_node_0.time] == 999999999999):
                self.array[3][Temp_node_0.time] = Temp_node_0
                Canpass_0 = True
            elif(heuristic < self.array[3][Temp_node_0.time].heuristic):
                node_to_change = self.array[3][Temp_node_0.time]
                to_change_predasessor = node_to_change.Predecessor

                if(to_change_predasessor.status == [0,0] and node_to_change.status == [0,0]):
                    to_change_predasessor.child[0] = None
                elif(to_change_predasessor.status == [0,0] and node_to_change.status == [1,0]):
                    to_change_predasessor.child[1] = None
                elif(to_change_predasessor.status == [1,0] and node_to_change.status == [0,1]):
                    to_change_predasessor.child[0] = None
                elif(to_change_predasessor.status == [1,0] and node_to_change.status == [1,1]):
                    to_change_predasessor.child[1] = None
                elif(to_change_predasessor.status == [1,1] and node_to_change.status == [0,1]):
                    to_change_predasessor.child[0] = None
                elif(
```

```
to_change_predasessor.status == [1,1] and node_to_change.status == [1,1]):
                    to_change_predasessor.child[1] = None
                elif(to_change_predasessor.status == [0,1] and node_to_change.status == [0,0]):
                    to_change_predasessor.child[0] = None
                elif(to_change_predasessor.status == [0,1] and node_to_change.status == [1,0]):
                    to_change_predasessor.child[1] = None

                self.array[3][Temp_node_0.time] = Temp_node_0
                for i in range(len(self.To_traverse)):
                    if(self.To_traverse[i] == node_to_change):
                        self.To_traverse.pop(i)
                        break
                Canpass_0 = True

        if(Canpass_0 == True):
            self.To_traverse.append(Temp_node_0)
            NodeTotraverse.child[0] = Temp_node_0

    #The last Node Is created and nothing is shift into the register
    elif(Temp_node_0.time == len(self.received)-1):
        heuristic = NodeTotraverse.heuristic + self.calculate_cost(self.received[Temp_node_0.time],Temp_node_0.getCval(NodeTotraverse.status, Temp_node_0.status))
        Temp_node_0.Predecessor = NodeTotraverse
        Temp_node_0.heuristic = heuristic

        Canpass_0 = False
        if(Temp_node_0.status == [0,0]):
            if(self.array[0][Temp_node_0.time] == 999999999999):
                self.array[0][Temp_node_0.time] = Temp_node_0
                Canpass_0 = True
            elif(heuristic < self.array[0][Temp_node_0.time].heuristic):
                node_to_change = self.array[0][Temp_node_0.time]
                to_change_predasessor = node_to_change.Predecessor

                if(to_change_predasessor.status == [0,0] and node_to_change.status == [0,0]):
                    to_change_predasessor.child[0] = None
```

```
                    elif(to_change_predasessor.status == [0,0] and node_to_change.status == [1,0]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [1,0] and node_to_change.status == [0,1]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [1,0] and node_to_change.status == [1,1]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [1,1] and node_to_change.status == [0,1]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [1,1] and node_to_change.status == [1,1]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [0,1] and node_to_change.status == [0,0]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [0,1] and node_to_change.status == [1,0]):
                        to_change_predasessor.child[1] = None

                    self.array[0][Temp_node_0.time] = Temp_node_0
                    for i in range(len(self.To_traverse)):
                        if(self.To_traverse[i] == node_to_change):
                            self.To_traverse.pop(i)
                            break
                    self.To_traverse.append(Temp_node_0)
                    Canpass_0 = True
            elif(Temp_node_0.status == [0,1]):
                if(self.array[1][Temp_node_0.time] == 999999999999):
                    self.array[1][Temp_node_0.time] = Temp_node_0
                    Canpass_0 = True
                elif(heuristic < self.array[1][Temp_node_0.time].heuristic):
                    node_to_change = self.array[1][Temp_node_0.time]
                    to_change_predasessor = node_to_change.Predecessor

                    if(to_change_predasessor.status == [0,0] and node_to_change.status == [0,0]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [0,0] and node_to_change.status == [1,0]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [1,0] and node_to_change.status == [0,1]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [1,0] and node_to_change.status == [1,1]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [1,1] and node_to_change.status == [0,1]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [1,1] and node_to_change.status == [1,1]):
                        to_change_predasessor.child[1] = None
                    elif(to_change_predasessor.status == [0,1] and node_to_change.status == [0,0]):
                        to_change_predasessor.child[0] = None
                    elif(to_change_predasessor.status == [0,1] and node_to_change.status == [1,0]):
                        to_change_predasessor.child[1] = None

                    self.array[1][Temp_node_0.time] = Temp_node_0
                    for i in range(len(self.To_traverse)):
                        if(self.To_traverse[i] == node_to_change):
                            self.To_traverse.pop(i)
                            break
                    Canpass_0 = True

            elif(Temp_node_0.status == [1,0]):
                if(self.array[2][Temp_node_0.time] == 999999999999):
                    self.array[2][Temp_node_0.time] = Temp_node_0
                    Canpass_0 = True
                elif(heuristic < self.array[2][Temp_node_0.time].heuristic):
                    node_to_change = self.array[2][Temp_node_0.time]
                    to_change_predasessor = node_to_change.Predecessor

                    if(to_change_predasessor.status == [0,0] and node_to_change.status == [0,0]):
```

```
to_change_predasessor.child[0] = None
                    elif(
to_change_predasessor.status == [0,0]
and node_to_change.status == [1,0]):

to_change_predasessor.child[1] = None
                    elif(
to_change_predasessor.status == [1,0]
and node_to_change.status == [0,1]):

to_change_predasessor.child[0] = None
                    elif(
to_change_predasessor.status == [1,0]
and node_to_change.status == [1,1]):

to_change_predasessor.child[1] = None
                    elif(
to_change_predasessor.status == [1,1]
and node_to_change.status == [0,1]):

to_change_predasessor.child[0] = None
                    elif(
to_change_predasessor.status == [1,1]
and node_to_change.status == [1,1]):

to_change_predasessor.child[1] = None
                    elif(
to_change_predasessor.status == [0,1]
and node_to_change.status == [0,0]):

to_change_predasessor.child[0] = None
                    elif(
to_change_predasessor.status == [0,1]
and node_to_change.status == [1,0]):

to_change_predasessor.child[1] = None

                    self.array[2][
Temp_node_0.time] = Temp_node_0
                    for i in range(len(
self.To_traverse)):
                        if(self.
To_traverse[i] == node_to_change):
                            self.
To_traverse.pop(i)
                            break
                    Canpass_0 = True

            elif(Temp_node_0.status ==
[1,1]):
                if(self.array[3][
Temp_node_0.time] == 999999999999):
                    self.array[3][
Temp_node_0.time] = Temp_node_0
                    Canpass_0 = True
                elif(heuristic < self.
array[3][Temp_node_0.time].heuristic):
                    node_to_change = self.
array[3][Temp_node_0.time]
                    to_change_predasessor
= node_to_change.Predecessor

                    if(
to_change_predasessor.status == [0,0]
and node_to_change.status == [0,0]):

to_change_predasessor.child[0] = None
                    elif(
to_change_predasessor.status == [0,0]
and node_to_change.status == [1,0]):

to_change_predasessor.child[1] = None
                    elif(
to_change_predasessor.status == [1,0]
and node_to_change.status == [0,1]):

to_change_predasessor.child[0] = None
                    elif(
to_change_predasessor.status == [1,0]
and node_to_change.status == [1,1]):

to_change_predasessor.child[1] = None
                    elif(
to_change_predasessor.status == [1,1]
and node_to_change.status == [0,1]):

to_change_predasessor.child[0] = None
                    elif(
to_change_predasessor.status == [1,1]
and node_to_change.status == [1,1]):

to_change_predasessor.child[1] = None
                    elif(
to_change_predasessor.status == [0,1]
and node_to_change.status == [0,0]):

to_change_predasessor.child[0] = None
                    elif(
to_change_predasessor.status == [0,1]
and node_to_change.status == [1,0]):

to_change_predasessor.child[1] = None

                    self.array[3][
Temp_node_0.time] = Temp_node_0
                    for i in range(len(
self.To_traverse)):
                        if(self.
To_traverse[i] == node_to_change):
                            self.
To_traverse.pop(i)
                            break
                    Canpass_0 = True

            if(Canpass_0 == True):
                NodeTotraverse.child[0] =
Temp_node_0


#Function to decode the built trellis
and return the shortest path
def MLSE_Decode(self):

    self.Root =  MNode([0,0] )
    self.Root.heuristic=0.0

    #Builds the trellis
    self.To_traverse = []
    self.To_traverse.append(self.Root)

    while(len(self.To_traverse)!= 0):
        NodeTotraverse = self.
```

```
    To_traverse.pop(0)
            self.buildTrellis(NodeTotraverse)


        #Gets the shortest path
        temp_path = []
        temp_path = self.getPath()

        transpose_arr = np.array(self.array)
        trs = transpose_arr.transpose()

        to_ret = []

        for i in range(len(temp_path)-2):

            if(i > 0):
                to_ret.append(temp_path[i])

        return to_ret

    #Function to Calculate the cost of the Node
    def calculate_cost(self, conv, conv_est):

        temp1 = abs(conv[0] - conv_est[0])**2
        temp2 = abs(conv[1] - conv_est[1])**2
        temp3 = abs(conv[2] - conv_est[2])**2
        sum = temp1 + temp2 + temp3
        return sum


# ----------------------
```

```
            bits1 = list(bits1)
            symbol1 = list(symbol1)
            if(constellation == '8psk'):
                bits1.pop()
                bits1.pop()
                s_1.pop()
                s_1.pop()

            ber+= quest3.calculate_BER(s_1,Q.MLSE_Decode())
            ber1+= quest3.calculate_BER(s_1,bits1)

        ber=ber/100
        ber1=ber1/100
        yvalues.append(ber)
        yvalues1.append(ber1)

    #Plotting
    plt.semilogy(xValues,yvalues, label="Convolution Encoder-MLSE using "+constellation)
    plt.ylabel('BER')
    plt.xlabel('SNR (dB)')

    plt.semilogy(xValues,yvalues1, label="Optimal detection uncoded "+constellation)

    plt.title(" BER vs SNR")
    plt.legend()
    plt.show()

#Function to test convelutional code with class example
def Test_MLSE():
    class_bits = [1,0,1,0,1,0,0]
    Encoded_bits = encoder(class_bits)
    print(Encoded_bits)

#Test_MLSE()
tester('bpsk')
tester('qpsk')
tester('8psk')
```

Appendix D

Question 3 Code

```python
import numpy as np
import random
import math
import time
import matplotlib.pyplot as plt

class q3:
    def __init__(self, numBits):
        self.numBits = numBits
        self.s1 = 10
        self.s2 = 20
        self.s3 = 30


    # Global seed variables


    def seed_unifrom(self, seed1:int, seed2:int, seed3:int)->None:
        """Advanced update of seed values for URNG
        :param seed1: the first seed value for WH Algorithm
        :param seed2: the second seed value for WH Algorithm
        :param seed3: the third seed value for WH Algorithm
        """
        self.s1 = seed1
        self.s2 = seed2
        self.s3 = seed3
    #end uniform()


    def uniform(self)->float:
        """Wichmann-Hill algorithm
        Use global variables
        """
        # Write your Wichmann-Hill algorithm code here
        #Declaration of constant primes
        constPrime1 = 40087
        constPrime2 = 50423
        constPrime3 = 61333

        #Declaration of arbitrary multipliers
        constMul1 = 347
        constMul2 = 346
        constMul3 = 348

        #Wichmann-Hill algorithm for updating the seeds
        self.s1 = (constMul1 * self.s1) % constPrime1
        self.s2 = (constMul2 * self.s2) % constPrime2
        self.s3 = (constMul3 * self.s3) % constPrime3

        #Restricting the result to [0,1)
        result = self.s1 / (constPrime1 * 1.0) + self.s2 / (constPrime2 * 1.0) + self.s3 /(constPrime3 * 1.0)
        result = result - math.trunc(result)
        return result
    #end uniform()

    global seed1
    global seed2
    seed1 = time.time()
    seed2 = time.time()+2

    def seed_gauss(self, s1:int, s2:int)->None:
        """Advanced update of seed values for GRNG
        :param seed1: the first seed value for GH Algorithm
        :param seed2: the second seed value for GH Algorithm
        """
        global seed1
        seed1 = s1
        global seed2
        seed2 = s2
        return

    def gauss(self)->float:
        """Marsaglia-Bray algorithm
        Use global variables
        """
        while True:
            global seed1
            global seed2
```

```
            seed1 += 1
            seed2 += 1
            random.seed(seed1)
            v1 = random.random() * 2.0 - 1.0
            random.seed(seed2)
            v2 = random.random() * 2.0 - 1.0

            if ((v1 * v1 + v2 * v2 < 1.0) and (v1 * v1 + v2 * v2 > 0.0)):
                break

            seed1 += 1
            seed2 += 2
        s = v1 * v1 + v2 * v2

        x1 = v1 * math.sqrt(-2.0 * math.log(s) / s)
        x2 = v2 * math.sqrt(-2.0 * math.log(s) / s)

        return random.choice([x1,x2])

    def generate_N_bits(self)->tuple:
        # Use your uniform random number generator to generate random bits. A value larger
        # than 0.5 is a 1 bit; a value less than or equal to 0.5 is a 0 bit. This will ensure a 50/50 0/1 bit ratio.
        """
            :param N: the number of values to generate
            :templis: temporary list to store values and later convert to tuple
            :return Tuplle containing random generated bits
        """
        templist = []
        for i in range(self.numBits):
            templist.append(int(np.round(self.uniform())))
        return tuple(templist) #convert list bit_stream into tuple and return
    #end generate_N_bits()

    def bits_to_symbols(self, bits:tuple, constellations='bpsk')->tuple:
        # Map the bits to symbols using the respective modulation constellations
        """
            :param bits: tuple of bits to be converted to symbols
            :param constellations: Constellation used for symbol conversion
            :temp: temporary list to store values and later convert to tuple
            :return Tuplle containing the converted symbols
        """
        temp = []
        if(constellations == 'bpsk'):
            for i in range(len(bits)):
                if(bits[i] == 1):#s1
                    temp.append(1)
                elif(bits[i] == 0):#s2
                    temp.append(-1)
        #end of constellations == 'bpsk'

        elif(constellations == '4qam'):
            if(len(bits) % 2 == 0): #if the number of is even
                i = 0
                while(i < len(bits) - 1):
                    if(bits[i] == 0 and bits[i + 1] == 0):
                        temp.append(round(1/2**0.5,4) + round(1/2**0.5,4)*1j) #s1

                    elif(bits[i] == 0 and bits[i + 1] == 1):
                        temp.append(round(-1/2**0.5,4) + round(1/2**0.5,4)*1j) #s2

                    elif(bits[i] == 1 and bits[i + 1] == 1):
                        temp.append(round(-1/2**0.5,4) + round(-1/2**0.5,4)*1j) #s3

                    elif(bits[i] == 1 and bits[i + 1] == 0):
                        temp.append(round(1/2**0.5,4) + round(-1/2**0.5,4)*1j) #s4

                    i += 2#post-increment index

            #else: #for the case when there's an odd number of bits in the stream
        #end of constellations == '4qam'

        elif(constellations == 'qpsk'):
            i = 0
            while(i < len(bits) - 1):
                if(bits[i] == 0 and bits[i + 1] == 0):
                    temp.append(1) #s1

                elif(bits[i] == 0 and bits[i + 1] == 1):
                    temp.append(1j)#s2

                elif(bits[i] == 1 and bits[i + 1] == 1):
                    temp.append(-1) #s3

                elif(bits[i] == 1 and bits[i + 1] == 0):
                    temp.append(-1j) #s4

                i += 2#post-increment index
```

```python


    elif(constellations == '8psk'):
        if(len(bits) % 3 == 0):
            i = 0
            while(i < len(bits) - 2):
                if(bits[i] == 1 and bits[i + 1] == 1 and bits[i + 2] == 1): #s1
                    temp.append(1)

                elif(bits[i] == 1 and bits[i + 1] == 1 and bits[i + 2] == 0): #s2
                    temp.append(round(1/2**0.5,4) + round(1/2**0.5,4)*1j)

                elif(bits[i] == 0 and bits[i + 1] == 1 and bits[i + 2] == 0): #s3
                    temp.append(1j)

                elif(bits[i] == 0 and bits[i + 1] == 1 and bits[i + 2] == 1): #s4
                    temp.append(round(-1/2**0.5,4) + round(1/2**0.5,4)*1j)

                elif(bits[i] == 0 and bits[i + 1] == 0 and bits[i + 2] == 1): #s5
                    temp.append(-1)

                elif(bits[i] == 0 and bits[i + 1] == 0 and bits[i + 2] == 0): #s6
                    temp.append(round(-1/2**0.5,4) + round(-1/2**0.5,4)*1j)

                elif(bits[i] == 1 and bits[i + 1] == 0 and bits[i + 2] == 0): #s7
                    temp.append(-1j)

                elif(bits[i] == 1 and bits[i + 1] == 0 and bits[i + 2] == 1): #s8
                    temp.append(round(1/2**0.5,4) + round(-1/2**0.5,4)*1j)

                i += 3 #post-increment index

        #else: #for the case when the number of bits in the stream isn't a multiple of 3
    #end of constellations == '8psk'

    elif(constellations == '16qam'):
        if(len(bits) % 4 == 0):
            i = 0
            while(i < len(bits) - 3):
                if(bits[i] == 1 and bits[i + 1] == 0 and bits[i + 2] == 0 and bits[i + 3] == 0): #(1000)S1
                    temp.append(round(1/2**0.5,4) + round(1/2**0.5,4)*1j)

                elif(bits[i] == 1 and bits[i + 1] == 0 and bits[i + 2] == 0 and bits[i + 3] == 1): #(1001)S2
                    temp.append(round(1/2**0.5,4) + round(1/8**0.5,4)*1j)

                elif(bits[i] == 1 and bits[i + 1] == 0 and bits[i + 2] == 1 and bits[i + 3] == 1): #(1011)S3
                    temp.append(round(1/2**0.5,4) + round(-1/8**0.5,4)*1j)

                elif(bits[i] == 1 and bits[i + 1] == 0 and bits[i + 2] == 1 and bits[i + 3] == 0): #(1010)S4
                    temp.append(round(1/2**0.5,4) + round(-1/2**0.5,4)*1j)

                elif(bits[i] == 1 and bits[i + 1] == 1 and bits[i + 2] == 0 and bits[i + 3] == 0): #(1100)S5
                    temp.append(round(1/8**0.5,4) + round(1/2**0.5,4)*1j)

                elif(bits[i] == 1 and bits[i + 1] == 1 and bits[i + 2] == 0 and bits[i + 3] == 1): #(1101)S6
                    temp.append(round(1/8**0.5,4) + round(1/8**0.5,4)*1j)

                elif(bits[i] == 1 and bits[i + 1] == 1 and bits[i + 2] == 1 and bits[i + 3] == 1): #(1111)S7
                    temp.append(round(1/8**0.5,4) + round(-1/8**0.5,4)*1j)

                elif(bits[i] == 1 and bits[i + 1] == 1 and bits[i + 2] == 1 and bits[i + 3] == 0): #(1110)S8
                    temp.append(round(1/8**0.5,4) + round(-1/2**0.5,4)*1j)

                elif(bits[i] == 0 and bits[i + 1] == 1 and bits[i + 2] == 0 and bits[i + 3] == 0): #(0100)S9
                    temp.append(round(-1/8**0.5,4) + round(1/2**0.5,4)*1j )

                elif(bits[i] == 0 and bits[i + 1] == 1 and bits[i + 2] == 0 and bits[i + 3] == 1): #(0101)S10
                    temp.append(round(-1/8**0.5,4) + round(1/8**0.5,4)*1j)

                elif(bits[i] == 0 and bits[i + 1] == 1 and bits[i + 2] == 1 and bits[i + 3] == 1): #(0111)S11
                    temp.append(round(-1/8**0.5,4) + round(-1/8**0.5,4)*1j)

                elif(bits[i] == 0 and bits[i + 1] == 1 and bits[i + 2] == 1 and bits[i + 3] == 0): #(0110)S12
                    temp.append(round
```

```
(-1/8**0.5,4) + round(-1/2**0.5,4)*1j)

                elif(bits[i] == 0 and bits[i + 1] == 0 and bits[i + 2] == 0 and bits[i + 3] == 0): #(0000)S13
                    temp.append(round(-1/2**0.5,4) + round(1/2**0.5,4)*1j)

                elif(bits[i] == 0 and bits[i + 1] == 0 and bits[i + 2] == 0 and bits[i + 3] == 1): #(0001)S14
                    temp.append(round(-1/2**0.5,4) + round(1/8**0.5,4)*1j)

                elif(bits[i] == 0 and bits[i + 1] == 0 and bits[i + 2] == 1 and bits[i + 3] == 1): #(0011)S15
                    temp.append(round(-1/2**0.5,4) + round(-1/8**0.5,4)*1j)

                elif(bits[i] == 0 and bits[i + 1] == 0 and bits[i + 2] == 1 and bits[i + 3] == 0): #(0010)S16
                    temp.append(round(-1/2**0.5,4) + round(-1/2**0.5,4)*1j)

                i += 4
        #else: #For the case when the number of transmitted bits isn't a multiple of 4
    #end constellation == '16qam'
    return tuple(temp)
#end bits_to_symbols()

def get_fbits(self, constellation='bpsk'):
    """
        This function returns the number of bits per
        symbol for a constellation.
    """
    if(constellation=='bpsk'):
        return 1
    elif(constellation == 'qpsk'):
        return 2
    elif(constellation == '4qam'):
        return 2
    elif(constellation == '8psk'):
        return 3
    elif(constellation == '16qam'):
        return 4
#end get_fbits()

def sigma(self, snr:float, f_bits:int, Rc:float):
    """
        This function computes and returns the sigma
        value used in the computation of symbol noise
        based on the number of bits per symbol in an
        encoding scheme and the signal to noise ratio.
    """
    return (1/math.sqrt((10**(snr/10))*2*f_bits*Rc))
#end sigma()

def add_noise(self, symbols:tuple, snr:float, f_bits:int, Rc:float)->tuple:
    """
        This function computes and adds noise to the
        symbols tuple passed to it based on the SNR and
        f_bits passed in its parameters.
    """
    temp = []
    sigma_x = self.sigma(snr, f_bits, Rc)
    for i in range(len(symbols)):
        random_gaus = self.gauss()
        to_add = complex(random_gaus, random_gaus)
        temp_val = symbols[i] + sigma_x*(to_add/math.sqrt(2))
        temp.append(temp_val)

    return tuple(temp)
#end add_noise()

def euclidean_distance(self, symbol, constellation='bpsk')->dict:
    # return the distances between the given symbol and every symbol in the given constellation
    # return your result as a dictionary, with the symbol as the key to its distance value
    # eg {<symbol_1>: distance_1, ..., <symbol_n>: distance_1 }
    # >>> eculidean_distance( 0.34, 'bpsk')
    # {1: 0.66 , -1: 1.34}
    if(constellation=='bpsk'):
        s1 = round(math.sqrt(abs(symbol - 1)**2),4)
        s2 = round(math.sqrt(abs(symbol + 1)**2),4)
        dictionarry = {1:s1, -1:s2}
        return dictionarry
    elif(constellation == '4qam'):
        s1 = round(math.sqrt(abs(symbol - complex(1/2**0.5, 1/2**0.5))**2),4)
        s2 = round(math.sqrt(abs(symbol - complex(-1/2**0.5, 1/2**0.5))**2),4)
        s3 = round(math.sqrt(abs(symbol - complex(-1/2**0.5, -1/2**0.5))**2),4)
        s4 = round(math.sqrt(abs(symbol - complex(1/2**0.5, -1/2**0.5))**2),4)

        dictionarry = {round(1/2**0.5,4) + round(1/2**0.5,4)*1j: s1, round(-1/2**0.5,4)
        + round(1/2**0.5,4)*1j: s2, round(-1/2**0.5,4) + round
```

```
(-1/2**0.5,4)*1j:s3,
            round(1/2**0.5,4) + round(-1/2**0.5,4)*1j:s4}
            return dictionarry

        elif(constellation == 'qpsk'):
            s1 = round(math.sqrt(abs(symbol - 1)**2),4)
            s2 = round(math.sqrt(abs(symbol - 1j)**2),4)
            s3 = round(math.sqrt(abs(symbol + 1)**2),4)
            s4 = round(math.sqrt(abs(symbol + 1j)**2),4)

            dictionarry = {1: s1, 1j: s2, -1:s3, -1j:s4}
            return dictionarry


        elif(constellation == '8psk'):
            s1 = round(math.sqrt(abs(symbol - 1)**2),2)
            s2 = round(math.sqrt(abs(symbol - complex(1/2**0.5, 1/2**0.5))**2),4)
            s3 = round(math.sqrt(abs(symbol - complex(0, 1))**2),2)
            s4 = round(math.sqrt(abs(symbol - complex(-1/2**0.5, 1/2**0.5))**2),4)
            s5 = round(math.sqrt(abs(symbol - (-1))**2),2)
            s6 = round(math.sqrt(abs(symbol - complex(-1/2**0.5, -1/2**0.5))**2),4)
            s7 = round(math.sqrt(abs(symbol - (complex(0, -1)))**2),2)
            s8 = round(math.sqrt(abs(symbol - complex(1/2**0.5, -1/2**0.5))**2),4)

            dictionarry = {1:s1, round(1/2**0.5,4) + round(1/2**0.5,4)*1j : s2, 1j : s3,
            round(-1/2**0.5,4) + round(1/2**0.5,4)*1j : s4, -1 : s5, round(-1/2**0.5,4) +
            round(-1/2**0.5,4)*1j : s6, -1j: s7, round(1/2**0.5,4) + round(-1/2**0.5,4)*1j : s8}
            return dictionarry

        elif(constellation == '16qam'):
            s1 = round(math.sqrt(abs(symbol - complex(1/2**0.5, 1/2**0.5))**2),4)
            s2 = round(math.sqrt(abs(symbol - complex(1/2**0.5, 1/8**0.5))**2),4)
            s3 = round(math.sqrt(abs(symbol - complex(1/2**0.5, -1/8**0.5))**2),4)
            s4 = round(math.sqrt(abs(symbol - complex(1/2**0.5, -1/2**0.5))**2),4)
            s5 = round(math.sqrt(abs(symbol - complex(1/8**0.5, 1/2**0.5))**2),4)
            s6 = round(math.sqrt(abs(symbol - complex(1/8**0.5, 1/8**0.5))**2),4)
            s7 = round(math.sqrt(abs(symbol - complex(1/8**0.5, -1/8**0.5))**2),4)
            s8 = round(math.sqrt(abs(symbol - complex(1/8**0.5, -1/2**0.5))**2),4)
            s9 = round(math.sqrt(abs(symbol - complex(-1/8**0.5, 1/2**0.5))**2),4)
            s10 = round(math.sqrt(abs(symbol - complex(-1/8**0.5, 1/8**0.5))**2),4)
            s11 = round(math.sqrt(abs(symbol - complex(-1/8**0.5, -1/8**0.5))**2),4)
            s12 = round(math.sqrt(abs(symbol - complex(-1/8**0.5, -1/2**0.5))**2),4)
            s13 = round(math.sqrt(abs(symbol - complex(-1/2**0.5, 1/2**0.5))**2),4)
            s14 = round(math.sqrt(abs(symbol - complex(-1/2**0.5, 1/8**0.5))**2),4)
            s15 = round(math.sqrt(abs(symbol - complex(-1/2**0.5, -1/8**0.5))**2),4)
            s16 = round(math.sqrt(abs(symbol - complex(-1/2**0.5, -1/2**0.5))**2),4)

            dictionarry = {round(1/2**0.5,4) + round(1/2**0.5,4)*1j : s1,
            round(1/2**0.5,4) + round(1/8**0.5,4)*1j :s2,
            round(1/2**0.5,4) + round(-1/8**0.5,4)*1j :s3,
            round(1/2**0.5,4) + round(-1/2**0.5,4)*1j :s4,
            round(1/8**0.5,4) + round(1/2**0.5,4)*1j :s5,
            round(1/8**0.5,4) + round(1/8**0.5,4)*1j: s6,
            round(1/8**0.5,4) + round(-1/8**0.5,4)*1j:s7,
            round(1/8**0.5,4) + round(-1/2**0.5,4)*1j :s8,
            round(-1/8**0.5,4) + round(1/2**0.5,4)*1j :s9,
            round(-1/8**0.5,4) + round(1/8**0.5,4)*1j :s10,
            round(-1/8**0.5,4) + round(-1/8**0.5,4)*1j :s11,
            round(-1/8**0.5,4) + round(-1/2**0.5,4)*1j :s12,
            round(-1/2**0.5,4) + round(1/2**0.5,4)*1j :s13,
            round(-1/2**0.5,4) + round(1/8**0.5,4)*1j :s14,
```

```
            round(-1/2**0.5,4) + round(-1/8**0.5,4)*1j :s15,
            round(-1/2**0.5,4) + round(-1/2**0.5,4)*1j :s16}
            return dictionarry
    #end euclidean_distance()

    def symbol_detection(self, symbols: tuple, constellation='bpsk') -> tuple:
        # return a tuple of the symbols based on the eculidean_distances results
        temp = []

        for i in range(len(symbols)):
            temp_key = ''
            value = 999999999

            temp_dic = self.euclidean_distance(symbols[i], constellation)

            for key in temp_dic:
                if temp_dic[key] <=  value:
                    value = temp_dic[key]

            for key in temp_dic:
                if temp_dic[key] == value:
                    temp.append(key)
                    break

        return tuple(temp)
    #end symbol_detection()

    def bit_detection(self, symbols:tuple, constellation='bpsk') -> tuple:
        """return a tuple of the bits based on the results of the symbols
        >>> bit_detection((1, -1, 1, -1), 'bpsk')
        (1, 0, 1, 0)
        >>> bit_detection((1, 1, 1, -1), 'bpsk')
        (1, 1, 1, 0)
        """
        temp = []
        decoded = []
        tempsymbols = self.symbol_detection(symbols, constellation)
        tempsymbols = list(tempsymbols)
        if(constellation == 'bpsk'):
            for i in range(len(tempsymbols)):
                if(tempsymbols[i] == 1):
                    temp.append(1)
                    decoded.append(1)

                elif(tempsymbols[i] == -1):
                    temp.append(0)
                    decoded.append(-1)
        #end of constellations == 'bpsk'

        elif(constellation == '4qam'):
            for i in range(len(tempsymbols)):
                if(tempsymbols[i] == round(1/2**0.5,4) + round(1/2**0.5,4)*1j):
                    temp.append(0)
                    temp.append(0)
                    decoded.append(-1)
                    decoded.append(-1)

                elif(tempsymbols[i] == round(-1/2**0.5,4) + round(1/2**0.5,4)*1j):
                    temp.append(0)
                    decoded.append(-1)
                    temp.append(1)
                    decoded.append(1)

                elif(tempsymbols[i] == round(-1/2**0.5,4) + round(-1/2**0.5,4)*1j):
                    temp.append(1)
                    temp.append(1)
                    decoded.append(1)
                    decoded.append(1)

                elif(tempsymbols[i] == round(1/2**0.5,4) + round(-1/2**0.5,4)*1j):
                    temp.append(1)
                    temp.append(0)
                    decoded.append(1)
                    decoded.append(-1)

                else:
                    print("hier is groot fout")


        #end of constellations == '4qam'
        elif(constellation == 'qpsk'):
            for i in range(len(tempsymbols)):
                if(tempsymbols[i] == 1):
                    temp.append(0)
                    temp.append(0)
                    decoded.append(-1)
                    decoded.append(-1)

                elif(tempsymbols[i] == 1j):
                    temp.append(0)
                    decoded.append(-1)
                    temp.append(1)
                    decoded.append(1)

                elif(tempsymbols[i] == -1):
                    temp.append(1)
                    temp.append(1)
                    decoded.append(1)
                    decoded.append(1)

                elif(tempsymbols[i] == -1j):
                    temp.append(1)
                    temp.append(0)
```

```
                    decoded.append(1)
                    decoded.append(-1)

        elif(constellation == '8psk'):
            for i in range(len(tempsymbols)):
                if(tempsymbols[i] == 1):
                    temp.append(1)
                    temp.append(1)
                    temp.append(1)
                    decoded.append(1)
                    decoded.append(1)
                    decoded.append(1)

                elif(tempsymbols[i] == round(1/2**0.5,4) + round(1/2**0.5,4)*1j): #s2
                    temp.append(1)
                    temp.append(1)
                    temp.append(0)
                    decoded.append(1)
                    decoded.append(1)
                    decoded.append(-1)

                elif(tempsymbols[i] == 1j): #s3
                    temp.append(0)
                    temp.append(1)
                    temp.append(0)
                    decoded.append(-1)
                    decoded.append(1)
                    decoded.append(-1)

                elif(tempsymbols[i] == round(-1/2**0.5,4) + round(1/2**0.5,4)*1j): #s4
                    temp.append(0)
                    temp.append(1)
                    temp.append(1)
                    decoded.append(-1)
                    decoded.append(1)
                    decoded.append(1)

                elif(tempsymbols[i] == -1): #s5
                    temp.append(0)
                    temp.append(0)
                    temp.append(1)
                    decoded.append(-1)
                    decoded.append(-1)
                    decoded.append(1)

                elif(tempsymbols[i] == round(-1/2**0.5,4) + round(-1/2**0.5,4)*1j): #s6
                    temp.append(0)
                    temp.append(0)
                    temp.append(0)
                    decoded.append(-1)
                    decoded.append(-1)
                    decoded.append(-1)

                elif(tempsymbols[i] == -1j): #s7
                    temp.append(1)
                    temp.append(0)
                    temp.append(0)
                    decoded.append(1)
                    decoded.append(-1)
                    decoded.append(-1)

                elif(tempsymbols[i] == round(1/2**0.5,4) + round(-1/2**0.5,4)*1j): #s8
                    temp.append(1)
                    temp.append(0)
                    temp.append(1)
                    decoded.append(1)
                    decoded.append(-1)
                    decoded.append(1)

        #end of constellations == '8psk'

        elif(constellation == '16qam'):
            for i in range(len(tempsymbols)):
                if(tempsymbols[i] == round(1/2**0.5,4) + round(1/2**0.5,4)*1j): #(1000)S1
                    temp.append(1)
                    temp.append(0)
                    temp.append(0)
                    temp.append(0)
                    decoded.append(1)
                    decoded.append(-1)
                    decoded.append(-1)
                    decoded.append(-1)

                elif(tempsymbols[i] == round(1/2**0.5,4) + round(1/8**0.5,4)*1j): #(1001)S2
                    temp.append(1)
                    temp.append(0)
                    temp.append(0)
                    temp.append(1)
                    decoded.append(1)
                    decoded.append(-1)
                    decoded.append(-1)
                    decoded.append(1)

                elif(tempsymbols[i] == round(1/2**0.5,4) + round(-1/8**0.5,4)*1j): #(1011)S3
                    temp.append(1)
                    temp.append(0)
                    temp.append(1)
                    temp.append(1)
                    decoded.append(1)
                    decoded.append(-1)
                    decoded.append(1)
                    decoded.append(1)

                elif(tempsymbols[i] == round(1/2**0.5,4) + round(-1/2**0.5,4)*1j): #(1010)S4
                    temp.append(1)
                    temp.append(0)
                    temp.append(1)
                    temp.append(0)
                    decoded.append(1)
                    decoded.append(-1)
                    decoded.append(1)
```

```
                    decoded.append(-1)

                elif(tempsymbols[i] == round(1/8**0.5,4) + round(1/2**0.5,4)*1j): #(1100)S5
                    temp.append(1)
                    temp.append(1)
                    temp.append(0)
                    temp.append(0)
                    decoded.append(1)
                    decoded.append(1)
                    decoded.append(-1)
                    decoded.append(-1)

                elif(tempsymbols[i] == round(1/8**0.5,4) + round(1/8**0.5,4)*1j): #(1101)S6
                    temp.append(1)
                    temp.append(1)
                    temp.append(0)
                    temp.append(1)
                    decoded.append(1)
                    decoded.append(1)
                    decoded.append(-1)
                    decoded.append(1)

                elif(tempsymbols[i] == round(1/8**0.5,4) + round(-1/8**0.5,4)*1j): #(1111)S7
                    temp.append(1)
                    temp.append(1)
                    temp.append(1)
                    temp.append(1)
                    decoded.append(1)
                    decoded.append(1)
                    decoded.append(1)
                    decoded.append(1)

                elif(tempsymbols[i] == round(1/8**0.5,4) + round(-1/2**0.5,4)*1j): #(1110)S8
                    temp.append(1)
                    temp.append(1)
                    temp.append(1)
                    temp.append(0)
                    decoded.append(1)
                    decoded.append(1)
                    decoded.append(1)
                    decoded.append(-1)

                elif(tempsymbols[i] == round(-1/8**0.5,4) + round(1/2**0.5,4)*1j ): #(0100)S9
                    temp.append(0)
                    temp.append(1)
                    temp.append(0)
                    temp.append(0)
                    decoded.append(-1)
                    decoded.append(1)
                    decoded.append(-1)
                    decoded.append(-1)

                elif(tempsymbols[i] == round(-1/8**0.5,4) + round(1/8**0.5,4)*1j): #(0101)S10
                    temp.append(0)
                    temp.append(1)
                    temp.append(0)
                    temp.append(1)
                    decoded.append(-1)
                    decoded.append(1)
                    decoded.append(-1)
                    decoded.append(1)

                elif(tempsymbols[i] == round(-1/8**0.5,4) + round(-1/8**0.5,4)*1j): #(0111)S11
                    temp.append(0)
                    temp.append(1)
                    temp.append(1)
                    temp.append(1)
                    decoded.append(-1)
                    decoded.append(1)
                    decoded.append(1)
                    decoded.append(1)

                elif(tempsymbols[i] == round(-1/8**0.5,4) + round(-1/2**0.5,4)*1j): #(0110)S12
                    temp.append(0)
                    temp.append(1)
                    temp.append(1)
                    temp.append(0)
                    decoded.append(-1)
                    decoded.append(1)
                    decoded.append(1)
                    decoded.append(-1)

                elif(tempsymbols[i] == round(-1/2**0.5,4) + round(1/2**0.5,4)*1j): #(0000)S13
                    temp.append(0)
                    temp.append(0)
                    temp.append(0)
                    temp.append(0)
                    decoded.append(-1)
                    decoded.append(-1)
                    decoded.append(-1)
                    decoded.append(-1)

                elif(tempsymbols[i] == round(-1/2**0.5,4) + round(1/8**0.5,4)*1j): #(0001)S14
                    temp.append(0)
                    temp.append(0)
                    temp.append(0)
                    temp.append(1)
                    decoded.append(-1)
                    decoded.append(-1)
                    decoded.append(-1)
                    decoded.append(1)

                elif(tempsymbols[i] == round(-1/2**0.5,4) + round(-1/8**0.5,4)*1j): #(0011)S15
                    temp.append(0)
                    temp.append(0)
                    temp.append(1)
                    temp.append(1)
                    decoded.append(-1)
                    decoded.append(-1)
                    decoded.append(1)
```

```
                    decoded.append(1)

                elif(tempsymbols[i] == round(-1/2**0.5,4) + round(-1/2**0.5,4)*1j): #(0010)S16
                    temp.append(0)
                    temp.append(0)
                    temp.append(1)
                    temp.append(0)
                    decoded.append(-1)
                    decoded.append(-1)
                    decoded.append(1)
                    decoded.append(-1)

        return tuple(temp), tuple(decoded)
    #end symbol_detection()

    def calculate_SER(self, transmitted:tuple, received:tuple)->float:
        """
            This function determines the symbol error rate bteween the transmitted and received
            symbols passed into the function as Tupples
        """
        counter = 0
        for i in range(len(transmitted)):
            if(transmitted[i] != received[i]):
                counter += 1

        total_transmitted = len(transmitted)
        return counter/total_transmitted
    #end calculate_SER()

    def calculate_BER(self, transmitted:tuple, received:tuple)->float:
        """
            This function determines the bit error rate bteween the transmitted and received
            bits passed into the function as Tupples
        """

        counter = 0
        for i in range(len(transmitted)):
            if(transmitted[i] != received[i]):
                counter += 1

        total_transmitted = len(transmitted)
        return counter/total_transmitted
    #end calculate_BER()

    plt.show()
```